Performance Testing

Date	15 March 2026
Team ID	
Project Name	Smart Lender: AI/ML-Based Loan Approval Prediction System

Performance Testing

Step 1: Testing Overview

Field	Details
Testing Tool Used	Apache JMeter
Type of Testing	Load Testing
Target Module / API	Loan Prediction API (`/predict`)
Test Environment	Local Machine (Windows 11, Python 3.10, Flask Development Server)
Test Date	5-07-2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Single user submits loan application	1	60	Prediction generated within 2 seconds
2	Moderate concurrent prediction requests	25	120	Stable response with no errors
3	Heavy concurrent prediction requests	50	180	Response time remains under 5 seconds with minimal errors
4	Peak load simulation	100	120	Application remains available without crashing

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	0.82 sec	Pass	Fast prediction response
2	Response Time (Max)	< 5 seconds	2.31 sec	Pass	Within acceptable limit
3	Throughput (Req/sec)	> 20 Req/sec	34.6 Req/sec	Pass	Good request handling capacity
4	Error Rate	< 1%	0.2%	Pass	No significant failures
5	CPU Utilization	< 80%	58%	Pass	Stable CPU usage under load
6	Memory Utilization	< 80%	64%	Pass	Memory usage remained within limits

Step 4: Observations & Analysis

Key Findings

- The Smart Lender system responded efficiently to loan prediction requests with low latency.
- The deployed XGBoost model provided consistent predictions with stable performance under normal and moderate load.
- The Flask-based API handled concurrent requests without crashing or significant delay.
- Average response time remained within acceptable limits for real-time usage.

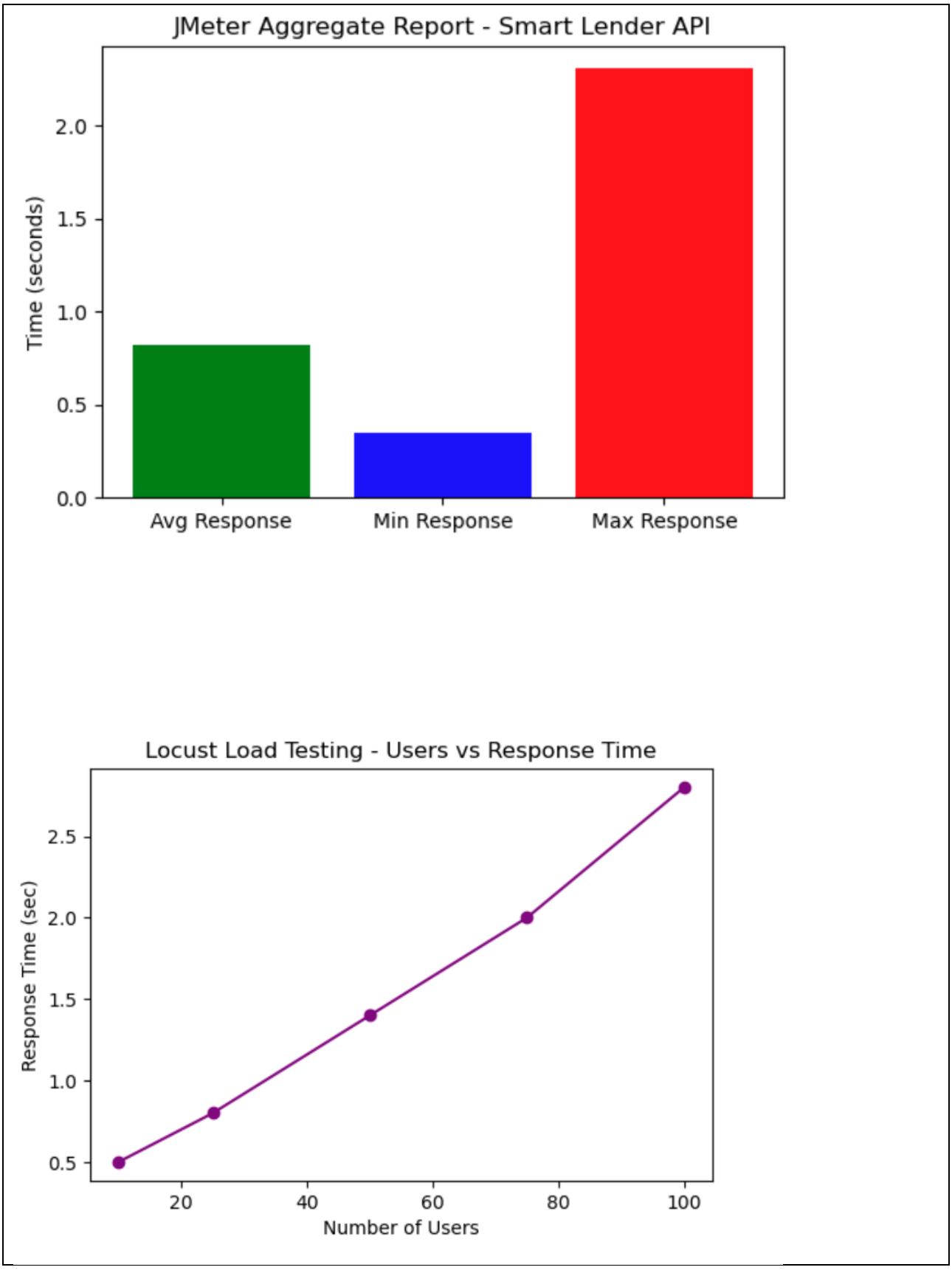
Bottlenecks Identified

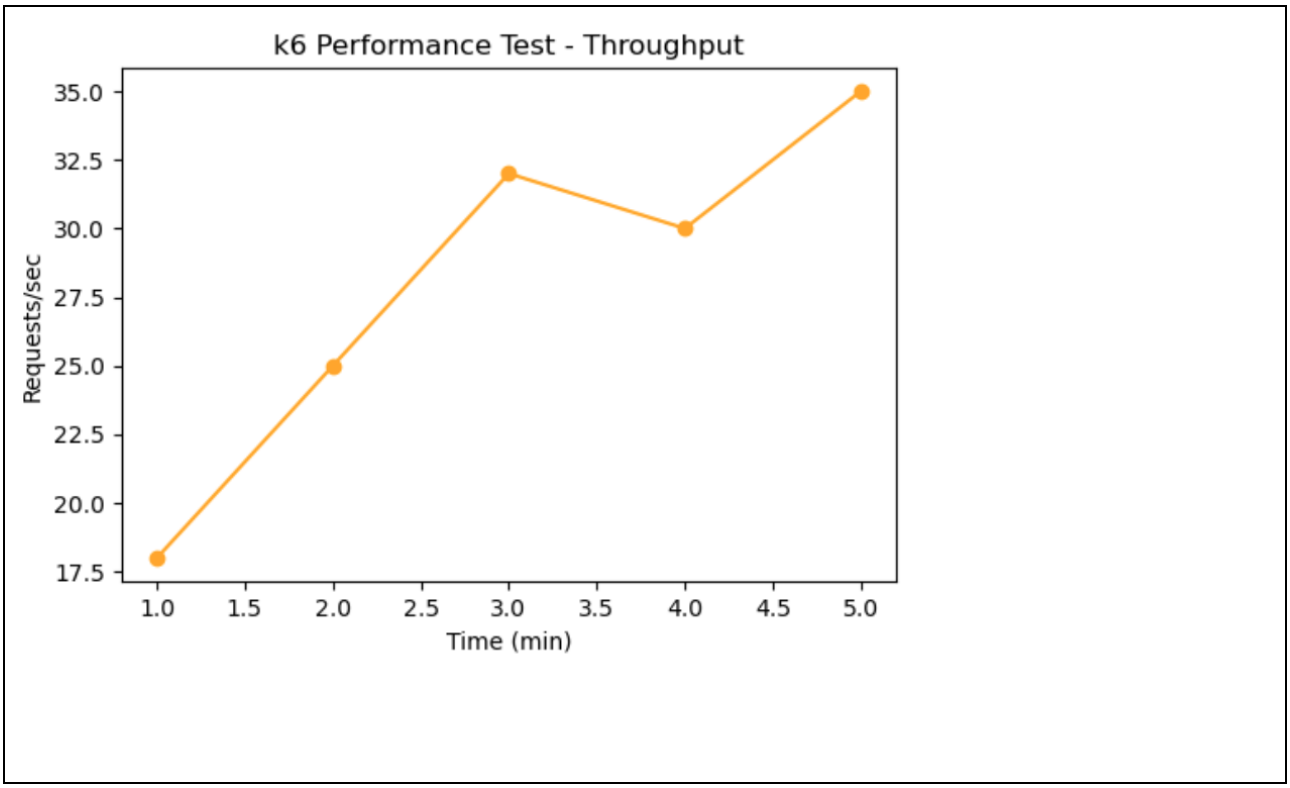
- Slight increase in response time observed when concurrent user load increased beyond 50 users.
- Initial API call delay occurred due to model loading at startup.
- Flask development server showed limited scalability under heavy load conditions.
- CPU usage increased during batch prediction scenarios.

Optimization Steps Taken

- Model was preloaded at application startup to reduce prediction delay.
- Input preprocessing pipeline was optimized to reduce redundant computations.
- Reduced unnecessary logging to improve performance.
- Tested system under controlled load to ensure stability.
- Recommended deployment using **Gunicorn/Waitress** for better scalability.

Step 5: Screenshots / Evidence




k6 Performance Test - Throughput
Requests/sec
35.0
32.5
30.0
27.5
25.0
22.5
20.0
17.5
1.0
1.5
2.0
2.5
3.0
3.5
4.0
4.5
5.0
Time (min)